Port.io Workflows

---

1. What Is Port.io?

Port is a developer portal and Internal Developer Platform (IDP) product. It helps engineering organisations create a central place where developers can discover software services, access platform capabilities, and execute approved self-service operations.

Port provides a software catalog and a workflow system that can coordinate operations across external tools such as GitHub, CI/CD platforms, cloud services, ticketing systems, and observability tools.

For example, consider a developer who wants to promote an application to a staging environment.

Without a self-service platform, the developer might need to:

1. Contact the platform engineering team.
2. Request the necessary configuration changes.
3. Wait for an engineer to update the repository.
4. Wait for a pull request to be created.
5. Follow the review and deployment process manually.

With a properly configured self-service workflow, the developer can submit a request through a portal. The workflow validates the request, invokes the required integrations, and returns the result.

The underlying Git, CI/CD, and deployment systems still perform their respective responsibilities.

1.1 The three main capabilities

1. Discover

Developers can discover services, teams, environments, ownership information, and other software-related resources through a software catalog.

2. Request

Developers can request approved operations through forms and self-service interfaces instead of manually coordinating every task.

3. Orchestrate

Platform engineers can connect validation, conditions, integrations, and external operations into workflows.

1.2 What Port is not

Port is not a replacement for every underlying development and deployment system.

For example:

- GitHub manages repositories and pull requests.
- CI/CD systems execute builds and tests.
- Flux CD or Argo CD reconciles Git-managed desired state.
- Kubernetes runs application workloads.
- Port coordinates operations across connected systems.

The central idea is that Port defines and governs workflows while connected systems perform the underlying operations.

---

2. Port as an Internal Developer Platform

2.1 The problem Port addresses

In many organisations, developers need to interact with several different systems to complete a single task.

For example, releasing an application might involve:

- A developer portal
- A Git repository
- A CI/CD system
- An approval process
- A deployment system
- Monitoring and observability tools

Without a common interface, developers must understand how these systems work individually.

Port helps provide a unified developer experience by bringing software information and self-service operations together.

2.2 The software catalog

Port's software catalog stores structured information about software entities.

Examples include:

- Services
- Teams
- Environments
- Deployments
- Repositories
- Other resources defined by the organisation

A **blueprint** defines the structure of a particular entity type. An entity is an individual record that follows that structure.

For example, an organisation could define a service blueprint containing:

- Service name
- Repository URL
- Owner
- Programming language
- Lifecycle
- Related deployment environment

A service entity might look conceptually like this:

```
None
Service: payments-api
    |
    ├── Owner: Payments Team
    |
    ├── Repository: GitHub repository
    |
    ├── Environment: Staging
    |
    └── Deployment status: Healthy
```

These fields are illustrative. Actual properties depend on how the organisation configures its catalog.

2.3 Why catalog context matters

A self-service workflow can use catalog information to understand which service a developer wants to modify.

Instead of asking developers to type every value manually, the portal can use known service information.

This can reduce mistakes and improve consistency.

However, catalog data must be maintained correctly. A workflow cannot reliably use incorrect or outdated ownership and deployment information.

Also, **catalog metadata is not a replacement for authorization checks**. A workflow must still verify that the requested operation is permitted.

---

3. Port Workflows Architecture

Port Workflows is a workflow orchestration system based on a graph of connected nodes.

A workflow defines:

1. How execution starts.
2. What information is received.
3. Which operations must be performed.
4. Which conditions determine the execution path.
5. What happens when operations succeed or fail.
6. How results are returned.

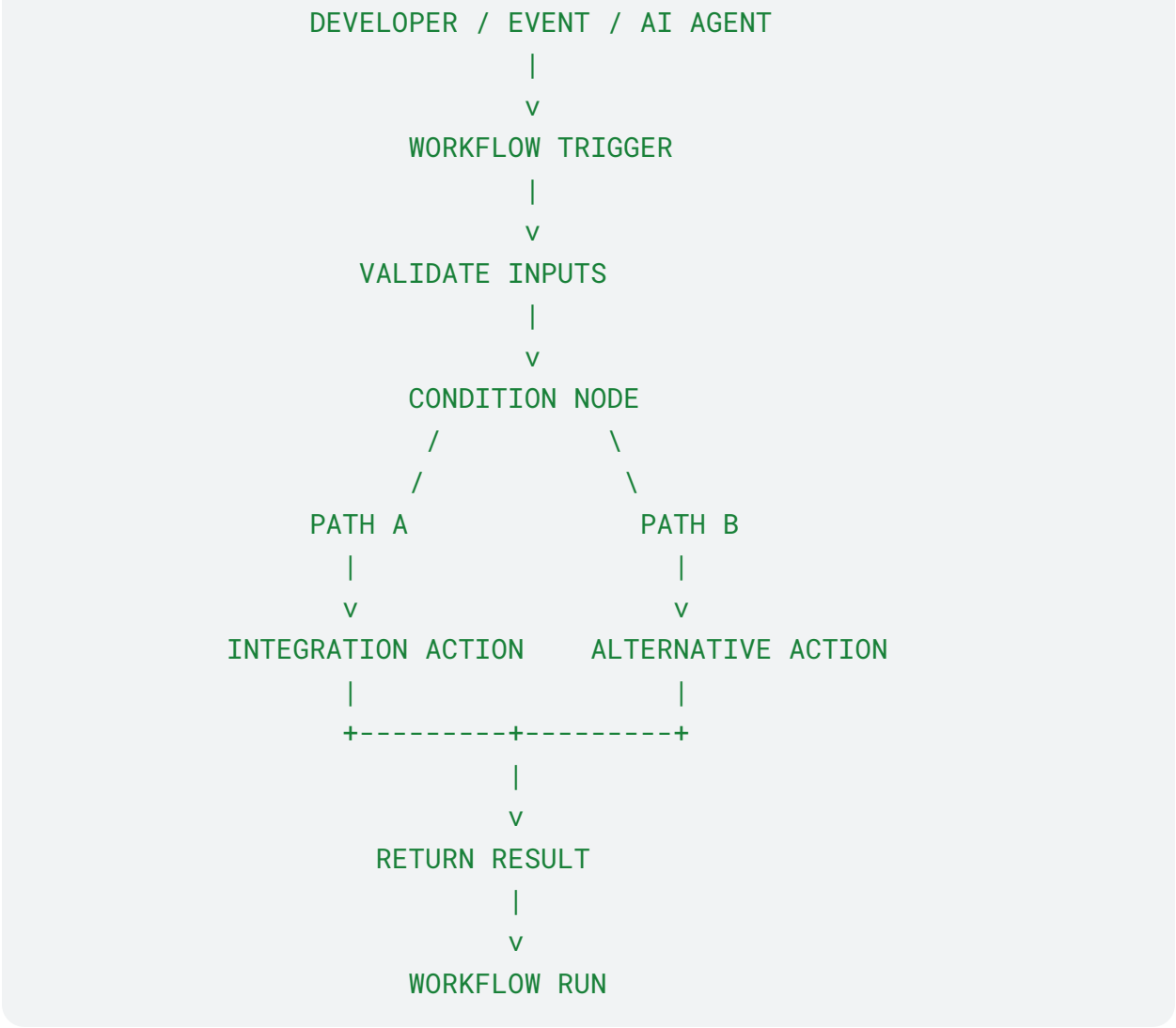
Port documents three fundamental node types:

- Trigger nodes
- Action nodes
- Condition nodes

Connections link nodes and determine the execution sequence.

3.1 General architecture

None



This is a conceptual diagram. A real workflow's structure depends on its configured nodes and connections.

3.2 Trigger nodes

A trigger starts the workflow.

It can provide initial data, such as a service identifier, target environment, or requested operation.

3.3 Action nodes

An action node performs an operation.

Examples include:

- Calling a supported integration action
- Creating a pull request
- Triggering a GitHub Actions workflow
- Sending a webhook request
- Performing another supported external operation

The available actions depend on Port's current capabilities and the configured integrations.

3.4 Condition nodes

A condition node evaluates an expression and selects an execution path.

For example:

```
None
Is the target environment production?
                |
          +----+----+
          |         |
         Yes        No
          |         |
          v         v
     Production   Standard
      policy      workflow
```

Port uses JQ expressions for workflow conditions.

3.5 Connections

Connections determine which node executes after another node.

They also enable outputs from previous nodes to be used by later nodes.

For example, a GitHub action might produce a pull request URL. A later node could use that URL in a notification or workflow result.

3.6 Workflow authoring

Port documents several approaches to creating workflows:

- A visual editor
- A JSON editor
- AI-assisted workflow creation

The visual editor is useful for understanding the execution graph. JSON can help when reviewing exact configuration. AI-assisted creation can speed up drafting, but the generated workflow should still be reviewed and tested.

A Port workflow is not necessarily a CI pipeline. It can coordinate multiple systems and operations, including invoking CI/CD pipelines where necessary.

---

4. Port Workflow Trigger Types

Port documents four trigger types.

These determine how a workflow can be started.

4.1 Human form or self-service trigger

A developer starts a workflow through a self-service interface.

For example:

```
None
Developer
    |
    v
Select component
    |
    v
Select target environment
    |
    v
Submit request
    |
    v
```

```
Workflow starts
```

Possible use cases include:

- Requesting an environment
- Promoting a release
- Triggering a redeployment
- Requesting an approved infrastructure change

For Project 702, this is similar to the experience you would build using a Backstage Scaffolder form.

The products use different implementations, but the developer-facing pattern is similar.

4.2 Catalog event trigger

A workflow can start when a catalog entity is created, updated, or deleted.

For example:

```
None
A new service entity is created
              |
              v
Workflow starts
              |
              v
Prepare onboarding tasks
              |
              v
Notify the service owner
```

This is useful when a catalog change should initiate an automated process.

4.3 Agent or MCP trigger

Port can expose self-service workflows through its Model Context Protocol (MCP) integration.

A compatible AI agent can discover available workflow tools and invoke an appropriate workflow.

For example:

```
None
Developer:
"Prepare a staging promotion for payments-api."

                |
                v

           AI Agent
                |
                v

      Discover available workflows
                |
                v

       Select promotion workflow
                |
                v

       Supply required parameters
                |
                v

          Execute workflow
```

The agent does not need to invent an unrestricted deployment procedure. It can invoke a predefined workflow with structured inputs.

MCP integration is an optional extension for Project 702, not a requirement for the MVP.

4.4 Schedule trigger

A scheduled trigger starts a workflow according to its configured schedule.

Examples include:

- Running routine audits
- Checking configuration consistency
- Performing scheduled maintenance
- Executing periodic housekeeping

These are examples of possible use cases, not claims that a particular schedule is enabled automatically.

4.5 Trigger comparison

Trigger	What starts it	Example
Human form	Developer submits a request	Promote a component
Catalog event	Catalog entity changes	Start service onboarding
Agent/MCP	Compatible agent invokes a tool	Request a staging deployment
Schedule	Configured schedule is reached	Run a periodic audit

Important design principles

- Keep inputs explicit and well-defined.
- Validate identifiers before performing changes.
- Define who is allowed to execute each workflow.
- Handle invalid inputs clearly.
- Do not assume every trigger is automatically enabled or secure without configuration.

---

5. Action Nodes, Conditions, and Connections

This section explains how Port turns a workflow definition into an executable process.

5.1 Action nodes

Action nodes perform the actual work.

For example, a workflow that creates a pull request might contain these steps:

```
None
1. Receive developer request
2. Validate inputs
3. Prepare repository change
4. Create pull request
5. Return pull request URL
```

Depending on the configured integration, actions can interact with GitHub, other supported source-control providers, external APIs, or webhooks.

The exact available actions depend on the current Port release and configured integrations.

5.2 Condition nodes

Condition nodes determine which path the workflow should follow.

Consider this example:

```
None
Developer requests a promotion
              |
              v
       Check target environment
              |
        +-----+-----+
        |           |
        v           v
   Development   Production
        |           |
        v           v
   Normal checks  Stricter checks
        |           |
        +-----+-----+
              |
              v
       Continue workflow
```

The production branch could require additional validation or approval.

This is a conceptual design; the actual policy and approval mechanisms must be configured.

5.3 JQ expressions

Port uses JQ expressions for conditions and data manipulation.

JQ is a language for querying and transforming JSON data.

For example, suppose a workflow receives this data:

```
None
{
  "component": "payments-api",
  "environment": "production"
}
```

A conceptual condition could check whether the environment equals `production`.

```
None
environment == "production"
```

The actual expression must use the data structure and expression context provided by the specific Port workflow node.

Do not assume this simplified expression can be pasted directly into every Port condition field.

5.4 Connections between nodes

Connections determine the order in which nodes execute.

For example:

```
None
Trigger
   |
   v
Validation
   |
   v
```

```
Condition
   |
   +---- Valid ----> Create PR
   |
   +---- Invalid --> Return error
```

A workflow should have a clear failure path.

If validation fails, the workflow should not silently continue to create a pull request.

5.5 Passing data between nodes

Suppose a GitHub action creates a pull request.

Its output might contain information such as the PR URL.

A later action can use that output to notify the developer.

```
None
Create GitHub PR
       |
       v
Receive PR result
       |
       v
Prepare notification
       |
       v
Return PR URL
```

This makes workflows more useful because later steps can depend on earlier results.

Key lesson: A workflow is more than a collection of independent actions. Its nodes are connected through execution order, conditions, and data flow.

---

6. Data Flow, Outputs, and JQ

Data flow determines how information moves through a workflow.

A workflow might receive input from a developer, retrieve additional context, perform an operation, and pass the result to another node.

6.1 Example input

A promotion workflow could receive:

```
None
{
  "component": "payments-api",
  "sourceEnvironment": "development",
  "targetEnvironment": "staging"
}
```

The workflow can use these values to determine which operation to perform.

6.2 Conceptual data flow

```
None
Developer input
       |
       v
Validate input
       |
       v
Determine target environment
       |
       v
Prepare repository change
       |
       v
Create pull request
       |
```

```
       v
Return PR information
```

6.3 Why data mapping matters

Data mapping helps a workflow:

- Pass developer input to external systems.
- Reuse results from previous actions.
- Make decisions based on runtime values.
- Construct requests for external APIs.
- Return useful information to the developer.

However, expression syntax depends on the particular action and its configuration. Use Port's current documentation when implementing actual workflow JSON or expressions.

6.4 Relevance to Project 702

Your Backstage Scaffolder implementation will also need to pass data between steps.

For example:

```
None
Scaffolder form
      |
      v
Component name
Environment
Release identifier
      |
      v
Validation action
      |
      v
occ generation
      |
      v
```

```
GitHub PR action
```

You do not need to copy Port's JQ implementation to achieve this.

Instead, use the existing Backstage Scaffolder mechanisms and the project's established coding patterns.

---

7. GitHub Integration and Pull Requests

Port supports integration actions for external systems, including GitHub.

The documented GitHub capabilities include operations such as triggering GitHub Actions workflows and creating or managing pull requests.

The exact available operations and requirements should be checked against the current integration documentation.

7.1 Typical GitHub workflow

```
None
Developer submits request
             |
             v
Port workflow starts
             |
             v
Validate request
             |
             v
Execute GitHub integration action
             |
             v
Create pull request
             |
             v
```

```
Return PR link
```

7.2 What must be configured?

A working GitHub integration requires appropriate configuration.

Important requirements include:

1. A GitHub integration configured in Port.
2. Access to the intended repository.
3. Permissions required by the requested operation.
4. Correct repository and branch information.
5. Appropriate credentials or integration authentication.
6. Any required GitHub Actions workflow configuration if a workflow is being triggered.

For self-hosted Port Ocean installations, action-processing configuration may also be necessary.

Do not assume an integration has all the permissions it needs simply because it has been connected.

7.3 PR creation is not deployment

This is one of the most important concepts for Project 702.

Consider the following sequence:

```
None
Workflow creates PR
        |
        v
Code review and CI checks
        |
        v
PR merged
        |
        v
GitOps controller detects change
        |
```

```
        v
Configuration reconciled
        |
        v
Application health evaluated
```

Each stage represents a different event.

Stage	Meaning
PR created	A proposed change exists.
PR approved	Required review conditions have been satisfied.
PR merged	The change has been accepted into the repository.
GitOps reconciled	The GitOps controller has processed the desired state.
Application healthy	Application health checks indicate a healthy state.

A successful PR creation does not prove deployment success.

Similarly, a merged PR does not guarantee that Flux successfully reconciled the change.

For Project 702, the portal should distinguish these statuses instead of reporting all of them as a single successful operation.

---

8. Self-Service Workflows

Self-service is one of the main reasons to use a developer portal.

Instead of manually performing routine operations, developers can request approved tasks through predefined workflows.

8.1 Common use cases

A. Service scaffolding

A developer requests a new service.

```
Developer
    |
    v
Submit service details
    |
    v
Validate inputs
    |
    v
Create repository or generate configuration
    |
    v
Return result
```

B. Environment provisioning

A developer requests an environment.

The workflow validates the request and invokes the appropriate infrastructure or automation system.

C. Deployment or redeployment

A developer requests a deployment operation for a particular application or version.

The workflow invokes the configured delivery system and returns the relevant status or execution link.

D. Rollback

A developer requests an approved rollback.

The workflow must validate the request and invoke the appropriate rollback mechanism. A real rollback implementation requires explicit safeguards.

E. Scaling an application

A developer requests a change to an application's replica count or another supported configuration value.

The workflow can validate the requested value and invoke the system responsible for applying it.

These are example use cases. Their availability depends on the integrations and workflows an organisation implements.

8.2 Characteristics of a good self-service workflow

A good workflow should:

1. Ask only for necessary information.
2. Use existing catalog or platform context where possible.
3. Validate inputs.
4. Enforce permissions.
5. Make important side effects clear.
6. Return meaningful results.
7. Handle failures explicitly.
8. Provide enough execution information for troubleshooting.

8.3 Why this matters for platform engineering

Platform engineering aims to make approved development tasks easier and more consistent.

A self-service workflow can hide unnecessary operational complexity without removing the controls required to operate a platform safely.

For Project 702, the developer should not need to understand every detail of OpenChoreo resource generation merely to request an approved promotion.

The platform should perform the appropriate generation and Git operations while preserving review and GitOps controls.

---

9. Permissions, Approvals, and Governance

Workflows that can modify repositories or trigger infrastructure operations require careful permission management.

A workflow is not secure merely because it is available through a portal.

9.1 Workflow execution permissions

Port documents permissions for controlling who can execute workflows.

These controls can involve roles, users, teams, and policies.

The documented default behaviour is that workflows without explicit execute permissions are restricted to administrators, who can execute workflows.

The precise permission configuration should be verified against the current Port release.

9.2 Workflow editing versus workflow execution

These are different permissions.

For example:

- A platform engineer may be allowed to modify a workflow definition.
- A developer may be allowed to execute the approved workflow.
- A developer may not be allowed to modify the underlying automation logic.

This separation is useful because it allows self-service without giving every developer permission to redefine privileged operations.

9.3 Execution privileges

Port documents that workflows execute with organisation-level permissions.

Consequently, a workflow can potentially perform operations beyond the privileges of the person who created it.

This makes careful configuration important.

For example, a developer might be permitted to request a staging promotion, while the workflow itself has access to a GitHub integration that can write to a repository.

The workflow must enforce the intended restrictions rather than assuming the integration's permissions will automatically enforce every business rule.

9.4 Example governance model

Operation	Possible control

Development promotion	Team membership and input validation
Staging promotion	Additional checks and repository review
Production promotion	Explicit approval and required branch checks
Workflow modification	Restricted workflow-edit permissions
Infrastructure changes	Strict validation and limited integration permissions
AI/MCP invocation	Explicitly permitted workflows and invocation policies

This table is a recommended design example, not a claim that Port automatically applies these exact policies.

9.5 Security principles

For workflows that perform privileged operations:

- Restrict who can invoke them.
- Validate all inputs.
- Use appropriately scoped integration credentials.
- Avoid exposing secrets in logs or workflow outputs.
- Enforce production controls.
- Test both normal and unauthorized execution paths.
- Review machine-token and AI/MCP permission behaviour before enabling agent execution.

For Project 702, GitHub branch protection and repository permissions remain important even when requests originate from Backstage.

---

10. Workflow Runs, Logs, and Status

A workflow is useful only if developers and platform engineers can understand what happened during execution.

Port documents workflow testing and run-level visibility, including step-level execution information and logs.

These features can help identify where an execution failed.

10.1 Why status separation matters

Consider a promotion workflow.

The workflow might successfully create a PR, but the PR may later fail its CI checks.

Alternatively, the PR may be merged successfully, but Flux may fail to reconcile the resulting configuration.

These are different failures and should be represented separately.

Status	What it tells you
Workflow succeeded	The configured workflow reached its success outcome.
PR opened	A proposed Git change exists.
PR merged	The change was accepted into the repository.
GitOps synced	The GitOps controller reports successful synchronization.
Application healthy	Relevant application health checks report a healthy state.

A workflow's success does not automatically prove that a downstream application is healthy.

10.2 Applying this to OpenChoreo

For Project 702, consider the following lifecycle:

```
None
Backstage request
       |
       v
Scaffolder task
       |
       v
GitHub PR
       |
       v
```

```
PR merge
       |
       v
Flux reconciliation
       |
       v
OpenChoreo resource status
       |
       v
Application health
```

The portal should expose the status information relevant to each stage.

For example, if the PR is merged but Flux fails, the developer should see that the Git change succeeded but deployment reconciliation failed.

That is much more useful than a generic failure message.

---

11. MCP and AI Agents in Port

Port's MCP integration allows compatible AI agents to discover and invoke exposed self-service workflows.

MCP stands for **Model Context Protocol**. It provides a standard mechanism through which compatible AI clients can discover and use tools.

The important idea is that an AI agent can interact with predefined platform operations rather than having to invent every operation itself.

11.1 How the process works

```
None
Developer asks an AI agent to perform a task
                    |
                    v
              AI agent
```

```
                |
                v
    Discover available workflow tools
                |
                v
     Read workflow inputs and description
                |
                v
      Select the appropriate workflow
                |
                v
         Supply required inputs
                |
                v
            Invoke workflow
                |
                v
      Workflow validates the request
                |
                v
         Execute configured actions
                |
                v
        Return result to the agent
```

Port's documentation identifies tools including:

- `list_self_service_triggers` — discover available self-service workflow triggers.
- `trigger_run` — invoke a workflow run.

The exact tool schemas and behaviour should be checked against the current Port documentation.

11.2 Example

A developer asks:

> Prepare a staging promotion for payments-api.

Conceptually, the agent could identify a suitable promotion workflow, collect the required parameters, and invoke it.

The workflow then applies its own validation and execution logic.

The agent should not bypass the workflow's safeguards simply because it is operating through natural language.

11.3 Security considerations

AI integration does not remove the need for access control.

Port documents different permission behaviour for machine tokens and personal access tokens. Machine-token execution may bypass some static role, user, or team checks by default, while policy checks can still be used to restrict invocation.

Therefore, before enabling AI-driven operations, the platform team must understand the applicable token permissions and invocation policies.

11.4 Relevance to Project 702

MCP could eventually provide an additional way to request OpenChoreo operations.

For example:

```
None
AI agent
   |
   v
Approved promotion operation
   |
   v
Backstage-compatible automation
   |
   v
GitHub PR
   |
   v
GitOps deployment
```

However, **MCP is not required for your Project 702 MVP**.

First, implement reliable deterministic operations, PR creation, and lifecycle status tracking. An AI interface can be considered later.

---

12. Software Catalog and Workflow Context

Port's software catalog provides structured information that workflows can use as context.

For example, a service entity might contain:

- Service identifier
- Repository URL
- Owner
- Programming language
- Lifecycle
- Related environment
- Deployment information

A workflow can use this information to make its execution more context-aware.

12.1 Example

Suppose the catalog contains:

```
None
Service: payments-api
Repository: GitHub repository
Owner: Payments Team
Lifecycle: Production
Environment: Staging
```

A developer selects `payments-api` in a self-service form.

The workflow can use the selected service's known metadata rather than asking the developer to enter every field again.

It can then validate the requested operation and invoke the appropriate integration.

12.2 Important limitations

The catalog is useful only if its data is accurate and maintained.

A workflow should not assume that every property is populated or that every catalog record is current.

It should also avoid using catalog information as the only security control.

12.3 Relevance to OpenChoreo

OpenChoreo already has its own concepts for projects, components, environments, releases, and deployment resources.

For Project 702, these should remain the authoritative platform concepts.

Port's catalog is a useful reference for understanding how structured resource information can improve self-service workflows. It does not mean you need to introduce a separate Port catalog into OpenChoreo.

---

13. Practical Example: A PR-Driven Promotion Workflow

This section combines the previous concepts into a single practical example.

Suppose a developer wants to promote `payments-api` from development to staging.

The intended result is a GitHub pull request containing the appropriate configuration change.

This is a **conceptual workflow design**, not a ready-to-import Port configuration.

13.1 Step 1: Define the inputs

The self-service form could request:

Input	Purpose
Component	Identifies the component to promote

Source environment	Identifies the current environment
Target environment	Identifies the destination
Release identifier	Identifies the intended release
Reason	Records why the change is requested

The actual fields depend on the platform's resource model.

13.2 Step 2: Validate the request

The workflow should verify that:

- Required inputs are present.
- The component exists.
- The source and target environments are valid.
- The release identifier is valid.
- The requested operation is permitted.
- No conflicting change prevents safe execution.

13.3 Step 3: Check policy

The workflow determines whether the requested promotion is permitted.

For example:

```
None
Promotion requested
         |
         v
   Check policy
         |
     +---+---+
     |       |
     v       v
  Allowed  Denied
     |       |
     v       v
Continue   Return reason
```

A production promotion might require additional controls.

The actual policy depends on the organisation's rules.

13.4 Step 4: Prepare the Git change

The workflow invokes the appropriate repository operation.

In a Port implementation, this would use configured actions or connected automation.

In Project 702, the implementation should use OpenChoreo-aware resource generation and the existing Backstage/GitHub integration.

13.5 Step 5: Create the pull request

The generated change is committed to a branch and submitted as a pull request.

A conceptual PR might contain:

```
None
Title:
Promote payments-api to staging

Summary:
- Component: payments-api
- Source environment: development
- Target environment: staging
- Release: <release-identifier>

This PR was generated by the self-service
promotion workflow.

Please review the generated configuration
and required checks before merging.
```

The exact resource changes must be generated according to the real repository structure and OpenChoreo resource model.

13.6 Step 6: Return the result

The workflow should return useful information, such as:

- Whether the request was accepted.
- Whether the PR was created.
- The PR URL.
- Any validation errors.
- The relevant workflow execution information.

13.7 Step 7: Track deployment after the PR

After the PR is merged, the workflow or associated status-tracking mechanism should correlate the change with the GitOps deployment lifecycle.

```
None
Pull request merged
         |
         v
Identify merged revision
         |
         v
Check Flux reconciliation
         |
         v
Check OpenChoreo resource status
         |
         v
Check application health
         |
         v
Update developer-facing status
```

This last step is essential.

Creating the PR is not the same as successfully deploying the application.

---

14. Mapping Port to OpenChoreo Project 702

Your project is titled:

Project 702: PR-Driven Self-Service GitOps for OpenChoreo Developers

The aim is to enable developers to request supported deployment changes through the existing OpenChoreo developer portal, with those changes represented as reviewable Git pull requests.

The supplied Project 702 repository review identifies the Backstage plugins as the main implementation area for the MVP.

It also identifies existing `occ` generation capabilities and a GitHub PR action, while GitOps-specific Scaffolder actions and lifecycle status tracking require implementation.

14.1 Port versus OpenChoreo

Port concept	Project 702 equivalent or analogy	Important distinction
Developer portal	Existing OpenChoreo Backstage portal	No second portal is required for the proposed MVP.
Self-service form	Backstage Scaffolder form	Similar user experience, different implementation.
Action node	Custom Scaffolder action	Implement the required OpenChoreo-specific operation.
Condition node	Validation and branching in the Scaffolder workflow	Port's JQ syntax does not need to be copied.
Software catalog	OpenChoreo project, component, and environment information	Preserve OpenChoreo's existing resource model.
GitHub action	Existing GitHub PR integration	Verify its permissions and required inputs.

Workflow run status	Scaffolder task and subsequent lifecycle status	A task's completion does not prove deployment success.
MCP workflow exposure	Possible future AI-agent interface	Optional extension, not an MVP requirement.

14.2 The most important lesson from Port

Do not think of the feature as simply:

> Add a Create PR button.

Think of it as:

> Build a reusable self-service GitOps workflow that transforms a developer's request into a validated, reviewable Git change and provides visibility into the outcome.

This is a much stronger architecture because it covers the full process.

14.3 What should remain in OpenChoreo?

The project should continue using its existing components:

- Backstage for the developer portal.
- Scaffolder for the self-service workflow.
- `occ` for OpenChoreo-aware resource generation where appropriate.
- GitHub integration for pull request creation.
- Flux CD for GitOps reconciliation.
- OpenChoreo for its resource and deployment model.

Port is an architectural reference, not a mandatory dependency.

---

15. Recommended Architecture for Project 702

The following architecture reflects the project requirements described in your supplied documents.

None

```
                    OPENCHOREO PROJECT 702

Developer
   |
   v
Existing Backstage Portal
   |
   v
Scaffolder Template / Form
   |
   v
Input Validation
   |
   v
Authorization and Conflict Check
   |
   v
Custom GitOps Scaffolder Actions
   |
   +---- Invoke existing occ generation
   |
   +---- Generate OpenChoreo resources
   |
   +---- Prepare repository changes
   |
   v
Git Branch
   |
   v
Commit
   |
   v
GitHub Pull Request
   |
   +---- Code review
   |
```

```
   +---- CI checks
   |
   +---- Branch protection
   |
   v
PR Merge
   |
   v
Flux CD
   |
   v
OpenChoreo Control Plane
   |
   v
Kubernetes Workloads
   |
   v
Status Correlation
   |
   v
OpenChoreo Developer Portal
```

15.1 Component responsibilities

Backstage

Provides the existing developer-facing portal.

Developers should not need to learn an entirely separate portal just to request a supported OpenChoreo operation.

Scaffolder

Collects the required inputs and orchestrates the workflow.

It coordinates validation, resource generation, and PR publishing.

occ

Generates OpenChoreo-aware resources using the available CLI capabilities.

Reusing the existing generation mechanisms is preferable to introducing unrelated, manually constructed resource templates.

GitHub

Stores the proposed change and provides the pull request, review, and branch-protection mechanisms.

Flux CD

Observes the relevant Git source and reconciles the desired state.

OpenChoreo

Processes its resources through the existing platform architecture.

Status tracking

Connects the original request to the resulting PR, merged revision, GitOps reconciliation, and deployment state.

15.2 Important implementation requirements

1. Reuse existing resource generation.

Use the appropriate existing `occ` commands instead of independently generating OpenChoreo YAML without considering the resource model.

2. Verify the runtime environment.

The supplied repository review notes that `occ` is not already packaged in the Backstage backend image.

The implementation must therefore account for installing or otherwise making the executable available and invoking it safely.

3. Implement conflict detection.

Check repository state and relevant existing changes before publishing a new PR.

4. Reuse the existing GitHub integration.

Avoid implementing a separate PR mechanism unless there is a clear technical reason.

5. Prevent GitOps bypasses.

In GitOps mode, supported promotion and override operations should not silently modify the live control plane through direct API writes when the intended workflow requires a PR.

6. Track the complete lifecycle.

Correlate:

```
None
Developer request
      |
      v
Pull request
      |
      v
Merge commit / revision
      |
      v
Flux reconciliation
      |
      v
OpenChoreo resource status
      |
      v
Application health
```

---

17. Recommended Implementation Roadmap

For your six-month Project 702, I recommend implementing the functionality in phases.

Phase 1: PR-based promotion

Priority: MVP

Implement:

- Developer-facing Scaffolder form
- Input validation
- Environment validation
- Conflict detection
- `occ` integration
- OpenChoreo resource generation
- Git branch and commit preparation
- GitHub pull request creation
- Clear success and failure messages

The first goal is to prove that a valid developer request can generate the correct configuration and create a reviewable PR.

Phase 2: Lifecycle status tracking

Priority: Essential for a complete developer experience

Track:

- Scaffolder task result
- PR creation status
- PR review and merge status
- Relevant merge revision
- Flux reconciliation status
- OpenChoreo resource status
- Application health where available

This phase closes the gap between creating a PR and understanding what happened after it was merged.

Phase 3: Overrides through PRs

Implement the PR-based handling of supported environment-specific overrides.

The supplied OpenChoreo reference distinguishes between an immutable `ComponentRelease` snapshot and a `ReleaseBinding` associating a release with an environment.

Promotion and override operations should therefore be implemented according to the actual OpenChoreo resource model.

Do not assume that release-binding generation automatically handles every override.

Phase 4: Advanced automation

Consider additional capabilities after the core workflow is reliable:

- Additional self-service operations
- Richer policy checks
- Notifications
- Improved conflict handling
- More detailed status reporting
- MCP or AI-agent integration

MCP is optional. It should not delay the essential PR and GitOps workflow.

17.1 Definition of done for the MVP

The first workflow should satisfy these requirements:

- A valid request generates the expected configuration.
- The generated change is submitted as a pull request.
- Invalid requests fail clearly.
- Conflicting changes are handled safely.
- Generation failures do not produce false-success results.
- GitHub permission failures are reported correctly.
- The developer receives the PR URL.
- PR creation is distinguished from deployment success.
- Credentials are not exposed in logs or generated files.

---

18. Testing and Troubleshooting Checklist

The following scenarios should be tested during implementation.

Scenario	Expected behaviour	What to inspect
Valid promotion	Generate configuration and create a PR	Scaffolder logs, generated diff, PR URL
Missing component	Reject the request	Input validation and resource lookup

Invalid environment	Reject or apply the configured policy	Environment validation
Conflicting change	Stop unsafe publishing	Repository and PR state
`occ` execution failure	Report generation error	Binary availability, arguments, working directory, error output
GitHub permission failure	Report PR publishing error	Credentials, repository access, permissions
PR merged, Flux fails	Report successful merge but failed reconciliation	Flux status and Kubernetes events
Flux synced, application unhealthy	Show sync and application health separately	Workload status, probes, logs
Duplicate request	Avoid duplicate or conflicting PRs	Existing PR lookup and request identity
Unauthorized operation	Deny the operation	Portal/backend authorization and repository policies

18.1 Recommended debugging sequence

When something fails:

1. Identify the failed stage.
2. Inspect the relevant task and execution logs.
3. Check the generated configuration.
4. Verify repository access and permissions.
5. Inspect the pull request and merge state.
6. Check Flux reconciliation.
7. Inspect the relevant OpenChoreo resources and workload health.
8. Return an accurate status to the developer.

This sequence helps distinguish application problems from workflow, GitHub, or GitOps problems.

---

19. Port vs Plural vs Harness

Since you previously studied Plural PR Automation, comparing these systems helps explain which ideas are most relevant to Project 702.

The following is a conceptual architectural comparison, not a guarantee that every product version supports every capability listed.

Dimension	Port	Plural	Harness
Central concept	Software catalog and workflow orchestration	PR Automation and platform/GitOps management	CI/CD, delivery orchestration, and GitOps capabilities
Developer entry point	Port portal and self-service forms	Plural Console and PR Automation interface	Harness platform and configured pipelines
Automation model	Graph of triggers, actions, and conditions	`PrAutomation` and related automation mechanisms	Pipeline stages, steps, and GitOps configuration
Git and PR operations	Supported integrations and actions	PR automation is a core pattern	Available through configured integrations and workflows
Software catalog	Central part of the developer experience	Platform and resource context	Services, environments, infrastructure, and delivery configuration
GitOps relationship	Orchestrates connected systems; does not replace Flux or Argo CD	GitOps-oriented platform management	Harness GitOps is built around Argo CD

AI-related capabilities	Documented workflow exposure through MCP	Documented AI-assisted PR automation	Depends on the relevant Harness products and integrations
Main lesson for Project 702	Self-service workflow, context, permissions, and status visibility	Transform developer intent into a Git change and PR	Delivery orchestration, verification, and governance

19.1 What should you learn from each system?

From Port

Learn how to structure self-service workflows, validate inputs, use contextual information, apply permissions, and provide visibility into execution.

From Plural

Learn the PR-driven automation pattern:

```
None
Developer intent
       |
       v
Automation definition
       |
       v
Generate or modify configuration
       |
       v
Create PR
       |
       v
Review and merge
       |
       v
GitOps reconciliation
```

From Harness

Study how delivery orchestration, verification, approvals, and governance can fit into a larger software delivery process.